EvoTune: Learning from Feedback to Guide LLMs for Slow-Query Tuning (Appendix)

Zhaoyan Hong[†], Xinyi Zhang[†], Zhentao Han[†], Wei Lu[†], Jinhao Dong[†],
Kai Xu[‡], Heng Long[‡], Liu Tang[‡], Qi Liu[‡], Xiaoyong Du[†]
[†]Renmin University, China [‡]PingCAP, China

OUTLINE

This supplemental material is organized as follows.
A. Prompt Templates for EvoTune Agents.
B. Experience Construction.
C. LLM Overhead and Tuning Efficiency.
D. Limitation of EvoTune.

A Prompt Templates for EvoTune Agents

This section presents the prompt templates used by the Detection Agent and Tuning Agent in EvoTune.

A.1 Detection Agent Prompt

Listing 1 shows the prompt template used by the Detection Agent. It takes the current query state, database environment information, and query-specific tuning history as input to identify promising tuning components.

Listing 1: Prompt template for tuning space localization.

Template: Detection Agent

You are a database performance diagnosis expert. Your task is to identify the most promising tuning component(s) for the following slow SQL query. To do so, you should reason over the current query state, database environment information, and query-specific tuning history, guided by DBA-style knowledge, and determine which components are most likely to deliver substantial performance improvement.

Candidate Tuning Components
The allowed components are exactly: [index, knob, hint, rewrite]

Context
1. Query State
- SQL Statement: `<SQL_QUERY>`
- Execution Plan: `<EXECUTION_PLAN_JSON>`
- Execution Runtime: `<EXECUTION_RUNTIME>`
- Internal Metrics: `<DBMS_SPECIFIC_METRICS>`

2. Database Environment Information
- Database Type: `<DB_TYPE>`
- Workload Type: `<WORKLOAD_TYPE>`
- Database Size: `<DATABASE_SIZE>`
- Database Schema: `<DATABASE_SCHEMA>`
- CPU: `<CPU_USAGE>`
- Read I/O: `<READ_IO>`
- Write I/O: `<WRITE_IO>`
- Virtual Memory: `<VIRTUAL_MEMORY>`
- Physical Memory: `<PHYSICAL_MEMORY>`
- Net Received: `<NET_RECEIVED>`
- Net Sent: `<NET_SENT>`

Reasoning Guidance
1. Query Tuning History
`<QUERY_TUNING_HISTORY>`

2. Trajectory-Guided Reasoning
Guide component reasoning according to the following principles:
- If `success_count>0` and `fail_count=0`: prioritize the component, as past interventions provide positive evidence of its effectiveness.
- If `success_count=0` and `fail_count>0`: deprioritize the component and encourage exploration of alternative components, as previous interventions involving this component have not yielded improvements.
- If `success_count>0` and `fail_count>0`: keep the component under consideration and make a more nuanced judgment based on DBA knowledge and the available evidence.

3. DBA Knowledge-Guided Reasoning
Use DBA-style knowledge to reason over the query context to identify target tuning components.
- **Index bottlenecks**: Index bottlenecks are characterized by large-range scans on selective predicates. Typical evidence includes a large gap between scanned and returned rows, predicates that cannot be effectively exploited by indexes, and repeated heap/table access due to non-covering indexes...
- **Knob misconfiguration**: Knob tuning becomes important when analytical operators spill to disk or lack sufficient memory or parallel resources. Typical evidence includes disk-backed temporary tables, underutilized parallel workers, and high-cost join or aggregation execution under limited resources...
- **Plan suboptimality**: Suboptimal access paths, join orders, or join strategies can cause performance bottlenecks that may be addressed through optimizer hints. Typical evidence includes cases where the optimizer prefers sequential scans despite the presence of indexes under high selectivity, incurs extra sorting due to unordered access paths...
- **Query formulation issues**: Query rewriting is prioritized when the SQL structure itself introduces unnecessary work. Typical evidence includes implicit type conversions (blocking B-Tree access and triggering full scans) and overlong `IN` lists...

Constraints (STRICT)
- You MUST select tuning components only from the following labels: `index`, `knob`, `hint`, `rewrite`.
- You MUST output strict JSON only, containing a single field `predicted_components`. No extra text and no explanation.
- `predicted_components` MUST be an array of the final selected tuning components.

Output JSON Format:
```
{"predicted_components": ["index","knob"]}
```

A.2 Tuning Agent Prompt

Listing 2 presents the prompt template used by the Tuning Agent. It combines the basic tuning context, localized tuning components, and the retrieved tuning demonstration to generate tuning actions.

Listing 2: Prompt template for action recommendation.

Template: Tuning Agent

You are an SQL optimization expert. The promising tuning components have already been localized. Your job is to recommend executable tuning action(s) **only within the localized tuning subspace**, using the current query context, localized tuning components, and the retrieved tuning demonstration (if available).

Context

1. Query State

- SQL: <SQL_QUERY>
- Execution Plan: <EXECUTION_PLAN_JSON>
- Execution Runtime: <EXECUTION_RUNTIME>
- Internal Metrics: <DBMS_SPECIFIC_METRICS>

2. Database Environment Information

- Database Type: <DB_TYPE>
- Workload Type: <WORKLOAD_TYPE>
- Database Size: <DATABASE_SIZE>
- Database Schema: <DATABASE_SCHEMA>
- CPU: <CPU_USAGE>
- Read I/O: <READ_IO>
- Write I/O: <WRITE_IO>
- Virtual Memory: <VIRTUAL_MEMORY>
- Physical Memory: <PHYSICAL_MEMORY>
- Net Received: <NET_RECEIVED>
- Net Sent: <NET_SENT>

3. Localized Bottleneck Component(s)

<TARGET_COMPONENTS>

4. Retrieved Tuning Demonstration

- Demonstration: <RETRIEVED_DEMONSTRATION>

5. Reflective Reasoning Guidance

- Query Tuning History: <QUERY_TUNING_HISTORY>

Before proposing tuning action(s), review the current query's tuning history and the retrieved demonstration to understand previous successes and failures. Adapt successful patterns while avoiding ineffective actions.

Constraints

- The tuning action must be directly executable in <DB_TYPE>.
- The tuning action must only operate on the component(s) listed in <TARGET_COMPONENTS>.
- The tuning action should be safe and must not drop or delete data.

Output Format

You must output *only* the following format:

```
Tuning_Action:<executable tuning action(s)>
Query_Rewrite:<yes or no>
```

Output Requirements

- Set `Query_Rewrite` to `yes` if the selected action(s) involve query rewriting, and place the full rewritten SQL in `Tuning_Action`; otherwise, set `Query_Rewrite` to `no`.
- If multiple executable statements are needed, place all of them in `Tuning_Action` in executable form.

B Experience Construction

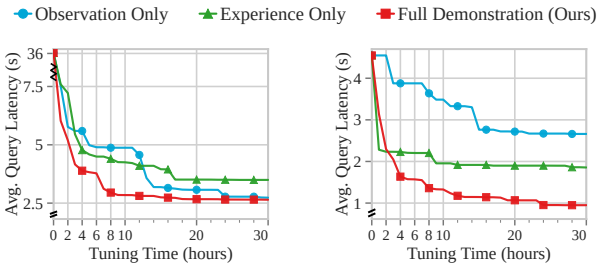
During tuning, EvoTune records tuning observations that capture information from each tuning attempt (including query states, localized components, applied actions, and execution feedback). While these observations provide fine-grained tuning evidence, they remain tied to specific queries and execution conditions, limiting their transferability across queries. To facilitate broader knowledge reuse, EvoTune further abstracts related observations into cluster-level experiences.

B.1 Experience Distillation and Maintenance

To avoid noisy or overly case-specific summaries, EvoTune first clusters related observations and then distills each cluster into an experience that captures a stable tuning pattern, including representative execution characteristics, recurring actions, and their overall effects. To identify related observations, EvoTune constructs a joint embedding by concatenating the query embedding, the binary encoding of localized tuning components, the action embedding produced by a BERT encoder, and the relative latency improvement R defined in Eq. (4) of the main paper. Considering the heterogeneous and noisy nature of tuning observations, EvoTune adopts HDBSCAN [1] to identify clusters of varying densities without requiring a predefined number of clusters, while leaving noisy observations unassigned. For each cluster, EvoTune invokes a Summary Agent to distill the tuning actions and execution feedback of its member observations into a reusable experience. For example, EvoTune distills the following experiences from historical observations:

- *Positive experience*: When execution remains scan-dominated and the number of planned or launched workers has reached the limit of `max_parallel_workers_per_gather`, increasing this limit may improve performance by enabling additional parallel scan workers.
- *Negative experience*: When execution is dominated by repeated large-scale aggregations and rescans, and the relevant sort/hash operators already fit in memory without spilling to disk, increasing `work_mem` may provide little benefit, since the bottleneck is not memory-bound.

As new observations arrive during online tuning, EvoTune incrementally maintains existing clusters rather than reclustering the entire memory. For each cluster, EvoTune maintains a prototype, defined as the average embedding of its member observations, and a radius, defined as the maximum distance from the prototype to its members. A new observation is assigned to the nearest cluster if its distance to the prototype falls within the corresponding radius; otherwise, it is stored in a pending buffer. When the buffer reaches a predefined threshold, EvoTune applies HDBSCAN to discover emerging clusters and invokes the Summary Agent to distill their experiences. Each distilled experience is combined with the tuning observations in its cluster to construct tuning demonstrations. During online tuning, after the Retriever selects a demonstration O_t, EvoTune provides it to the Tuning Agent. This allows EvoTune to combine fine-grained observation evidence with more generalizable tuning guidance.



(a) TPC-H, Postgres **(b) JOB, Postgres**

Figure 1: Effectiveness of Demonstration Construction.

Table 1: Tuning performance and LLM overhead across different LLM backends on TPC-H (SF10).

Metric	GPT-4.1		GPT-6.1 Sol	
	EvoTune	Booster	EvoTune	Booster
Sum of Query Latencies (s)	52.01	105.06	49.60	77.56
Avg. Iterations to Best	18	24	13	21
Avg. Cost to Best ($/query)	0.339	0.314	0.300	0.291
Avg. Input Tokens (/iter)	7502	5159	9179	5387
Avg. Output Tokens (/iter)	229	135	300	240
Avg. API Cost ($/iter)	0.0168	0.0114	0.0214	0.0132

B.2 Effectiveness of Demonstration Construction

To evaluate the effectiveness of observation-level evidence and cluster-level experiences, we organize historical observations using HDBSCAN (*min_cluster_size* = 5, *min_samples* = 5), with a pending buffer of size 20 triggering periodic clustering of pending observations. We compare three variants: (1) *Observation Only*, which uses the observation fields of the retrieved demonstration without distilled experience; (2) *Experience Only*, which uses only its associated experience; and (3) *Full Demonstration (Ours)*, which provides both the observation and its associated experience.

As shown in Figure 1, *Full Demonstration* consistently achieves the lowest query latency across both workloads. On TPC-H, *Observation Only* outperforms *Experience Only*, indicating that fine-grained tuning evidence provides more effective guidance for this workload. In contrast, on JOB, *Experience Only* substantially outperforms *Observation Only*, highlighting the benefits of generalizable tuning patterns. Despite these differences, *Full Demonstration* consistently outperforms both alternatives, demonstrating that combining instance-level evidence with cluster-level experiences provides more robust and effective tuning guidance across workloads.

C LLM Overhead and Tuning Efficiency

To evaluate the LLM inference overhead and tuning efficiency of EvoTune, we compare it with Booster using GPT-4.1 and GPT-6.1 Sol on TPC-H workload. Table 1 reports tuning performance and API overhead across both LLM backends. Compared with Booster, EvoTune reduces total query latency by 50.5% and 36.0% with GPT-4.1 and GPT-6.1 Sol, respectively, while increasing the cumulative API cost to the best observed configuration by only 8.0% and 3.1%. Although EvoTune consumes more tokens per iteration, it reaches its best observed configuration in fewer iterations. These results show that EvoTune achieves better tuning outcomes with modest additional API expenditure.

D Limitation of EvoTune

EvoTune focuses on slow-query tuning, a common and labor-intensive task in database maintenance, where DBAs frequently handle tuning requests that require manual diagnosis and optimization. Index tuning plays an important role in addressing slow queries and is therefore included as one of EvoTune's tuning components. However, persistent index changes may also affect other queries because indexes are shared across queries. Although tuning is typically performed on a standby DBMS or during idle periods to avoid interfering with production workloads, indexes recommended by EvoTune still require DBA review to assess their potential impact on other queries before production deployment.

References

[1] Leland McInnes, John Healy, and Steve Astels. 2017. HDBSCAN: Hierarchical density based clustering. *Journal of Open Source Software* 2, 11 (2017), 205. doi:10.21105/joss.00205